

UNIDAD III

"Sentences are not a mixture of words. Sentences are articulated". (Wittgenstein)

EL PRETÉRITO INDEFINIDO (SIMPLE PAST)

Uso: Usamos el "Past Simple" para referirnos a acciones realizadas en situaciones pasadas.

MARCADORES TEMPORALES

Yesterday, a month/year ago, an hour ago, last night/week/month/year/century.

• Verbo "to be"

Forma: **was** (I / he / she / it); **were** (you / we / you / they)

Afirmativo:

- I was at the library yesterday.
- We were at home before coming to college.

Negativo:

- I was not (wasn't) at the library yesterday.
- We were not (weren't) at home before coming to college.

Interrogativo:

- Was I at the library yesterday?
- Were we at home before coming to college?

• VERBOS REGULARES E IRREGULARES (Ver apéndice III)

Algunos ejemplos:

	INFINITIVO	PASADO	PASADO PARTICIPIO	PRESENTE PARTICIPIO
Verbos Regulares (terminan en "-ed")	use	Used	used	using
	apply	applied	applied	applying
	pass	passed	passed	passing
	push	pushed	pushed	pushing
	need	needed	needed	needing
	part	parted	parted	parting
Verbos Irregulares (igual forma en Pasado y Pasado Participio)	set	set	set	setting
	cut	cut	cut	cutting
	put	put	put	putting
	read	read	read	reading
Verbos Irregulares (distinta forma en Pasado y Pasado Participio)	grow	grew	grown	growing
	make	made	made	making
	find	found	found	finding
	bring	brought	brought	bringing
	write	wrote	written	writing
	come	came	come	coming
	go	went	gone	going

🔑 PRÁCTICA

Lea el siguiente texto y realice la ejercitación que está a continuación:

Ghezzi Carlo Full professor with tenure

Curriculum Vitae

Carlo Ghezzi is a Professor and Chair of Software Engineering at Politecnico di Milano. He is the Rector's Delegate for research. In the past, he was a member of the Academic Senate and a Member of the Board of Governors of Politecnico. He also held positions as Department Chair, Head of the PhD Program and Head of "consiglio di corso di laurea". He was also affiliated with: Università di Padova, University of California at Los Angeles, University of North Carolina at Chapel Hill, University of California at Santa Barbara (USA), Escuela Superior Latino-Americana de Informatica (Argentina), Technical University of Vienna and University of Klagenfurt (Austria), University of Lugano (Switzerland).

He is a Fellow of the ACM (citation: Numerous research contributions from compiler theory to real-time systems to software processes. A strong contributor to the software engineering community in Europe and worldwide.) and Fellow of the IEEE (citation: Contributions to programming languages and software engineering). He was awarded the ACM SIGSOFT Distinguished Service Award. He is a member of the Academy of Sciences. He has been a member of several governmental committees and was the Italian representative in the EU Information Technology Committee (Esprit Programme) during the 4th Framework Programme. He has been on the board of several international research projects and institutions in Europe, Japan, and the USA.

He is a regular member of the program committee of flagship conferences of the software engineering field, such as the International Conference on Software Engineering and Foundations of Software Engineering/ European Software Engineering Conference, for which he also served as Program and General Chair. He was General Chair of the International Conference on Service Oriented Computing (ICSOC 2006, Chicago, USA). He was the Editor in Chief of the ACM Transactions on Software Engineering and Methodology and Associate Editor of IEEE Transactions on Software Engineering. He is currently an Associate Editor of Science of Computer Programming (Elsevier), Service Oriented Computing and Applications (Springer), and Software Process Improvement and Practice (J. Wiley and Sons).

Carlo Ghezzi's research has been focusing on different aspects of software engineering and programming languages. Currently, he is active in the area of software architectures, especially evolvable and distributed software architectures for ubiquitous and pervasive computer applications. He co-authored over 150 papers, almost all of which are published internationally, and 8 books. He coordinated several national and international (EU funded) research projects.



[Print this page](#)

Contacts

Campus: Edificio 22 - Via Golgi, 42
Floor: 3
Office: 302
Ph.: 3529
Fax.: 3574
E-mail: carlo.ghezzi@polimi.it

1. **Identifique los verbos en Pasado y brinde una versión en Español de las oraciones correspondientes.**
2. **Responda en Español:**
 - a- ¿Cómo se llama el personaje de la foto? ¿Qué información puede dar acerca del mismo? (Resuma)
 - b- ¿Reconoce su apellido? ¿Conoce alguna obra de su autoría? ¿De qué trata la misma?

→ **Ahora, lea el siguiente texto y responda las preguntas que están a continuación:**

Programming Language Concepts

Carlo Ghezzi, Mehdi Jazayeri

3rd edition, ISBN 0-471-10426-4

This is the home page for the Third Edition of the book *Programming Language Concepts* by Carlo Ghezzi and Mehdi Jazayeri, published by John Wiley & Sons. The book is now in its second printing. The first printing had some printing problems (poor margins) which have been corrected in the second printing. This site is intended to provide supporting material for instructors and students. As the preface of the book states, "Programming languages are at the heart of computer science. They are the tools we use to communicate with computers and also with people. The challenge of designing language features that support clear expression, the puzzle of fitting together different features to make a useful language, and the challenge of appropriately using those features for the clear expression of algorithms make up part of the excitement of the study of programming languages. This is the third edition of Programming Language Concepts. We started writing the first edition in 1980 when it appeared to us that the field of programming languages was becoming stable. We were way wrong then! To this day, the field continues to be dynamic and exciting. New concepts and languages continue to arrive to excite large communities of computer scientists, theorists, experimentalists, and practitioners alike. Indeed, programming languages represent one of the few areas of computer science where practitioners and theorists find common ground. Emerging technologies and new application requirements require continuous evolution and innovation in the field and are sure to keep the flow of ideas and concepts going for a long time. This book studies fundamental concepts that underlie the features of programming languages... "

<http://www.infosys.tuwien.ac.at/pl-book/>

1. ¿A qué corresponde el sitio cuya dirección está arriba?
2. ¿Qué dice la parte transcripta del Prefacio? (traduzca)

→ **Ahora ingrese a <http://www.infosys.tuwien.ac.at/pl-book/toc.html> y responda:**

1. ¿Qué contiene la página arriba señalada?
2. ¿En cuántos capítulos están divididos los contenidos del libro? ¿De qué trata cada uno de éstos capítulos? ¿Y las secciones en cada capítulo?

→ **Finalmente, lea las dos secciones siguientes y resuma el contenido de las mismas.**

1.6 A brief historical perspective (Chapter 1 - page 14)

This section examines the developments in language design briefly by following the evolution of ideas and concepts from a historical perspective.

The software development process originally consisted only of the implementation phase. In the early days of computing, the computer was used mainly in scientific applications. An application was programmed by one person. The problem to be solved (e.g., a differential equation) was well understood. As a result, there was not much need for requirements analysis or design specification or even maintenance. A programming language, therefore, only needed to support one programmer, who was programming what would be by today's standards an extremely simple application. The desire to apply the computer in more and more applications led to its being used in increasingly less understood and more sophisticated environments. This, in turn, led to the need for "teams" of programmers and more disciplined approaches. The requirements and design phases, which up to then essentially were performed in one programmer's head, now required a team, with the results being communicated to other people. Because so much effort and money was being spent on the development of systems, old systems could not simply be thrown away when a new system was needed. Economic considerations forced people to enhance an existing system to meet the newly recognized needs. Also, program maintenance of such "legacy" systems became an important issue.

System reliability is another issued that has gained importance gradually, because of two factors. One factor is that systems are being developed for users with little or no computer background; these users are not as tolerant of system failures as the system developers. The second factor is that systems are now being applied in critical areas such as chemical or nuclear power plants and patient monitoring, where system failures can be disastrous. In order to ensure reliability, verification and validation became vital.

The perceived shortcomings of programming languages have led to a great number of language design efforts. This book examines these influences on language design and assesses the extent to which the resulting languages meet their goals. Sections 1.6.1 through 1.6.6 describe the historical evolution of programming languages discussed in this book. The year associated with each language should be taken as indicative: depending on the availability of the relevant information, it may mean either the year(s) of the language design, of its initial implementation, or of its first available published material on the languages listed in Table 1, including standards, can be found in the appendix at the end of the book.

Note: For better understanding, see Table 1 from the book (page 13).

2 Syntax and semantics (Chapter 2- page 33)

A programming language is a formal notation for describing algorithms for execution by computer. Like all formal notations, a programming language has two major components: syntax and semantics. The syntax is a set of formal rules that specify the composition of programs from letters, digits, and other characters. For example, the syntax rules may specify that each open parenthesis must match a closed parenthesis in arithmetic expressions, and that any two statements must be separated by a semicolon. These semantic rules specify "the meaning" of any syntactically valid program written in the language. Such meaning can be expressed by mapping each language construct into a domain whose semantics is known. For example, one way of describing the semantics of a language is by giving a description of each language construct in English. Such a description, of course, suffers from the informality, ambiguity, and wordiness of natural language, but it can give a reasonably intuitive view of the language. To provide a complete formal description of language semantics, syntactically valid programs are mapped onto mathematical domains. We will provide a preliminary introduction to formal semantics in this chapter, although a full treatment of the subject is out of our scope. Rather, we will focus on a rigorous yet informal description of semantics by specifying the behaviour of an abstract processor that executes programs written in the language. This kind of

semantic characterization of a language is called *operational semantics*. It could be presented using a rigorous and formal notation. Instead, we will follow a more traditional and informal approach, because it is more easily and intuitively understood by computer programmers and provides a high-level view of the problems found in implementing the language.

This chapter is organized as follows: in section 2.1 we discuss how the syntax and semantics of a language can be defined. In Section 2.2 we discuss language implementation and introduce the fundamental semantic concept of binding. In Section 2.3 and Section 2.4 we discuss two important concepts of programming languages – variables and routines – and their binding properties. In Section 2.6 we define an abstract semantic processor that can be used to specify operational semantics. In Section 2.7 we discuss how the abstract processor implements the main run-time features of programming languages.

→ EL PASADO CONTINUO

Uso: Usamos el Pasado continuo (Past Continuous or Progressive) para referirnos a acciones progresivas realizadas en el pasado.

Estructura: **WAS / WERE + -ING**

e.g.: I **was reading** an interesting book when the phone **rang**.

🔑 PRACTICA

Traduzca al Español las siguientes oraciones:

- a- "What were you doing at this time yesterday?" "I was studying for an exam."
- b- "Was Carol at work last night?" "Yes, she was finishing her final report".
- c- When I was young, I wanted to be an Information Systems Engineer.
- d- An Engineer was studying a dam when he was suddenly killed.
- e- Jane was waiting for me at the company when I arrived.
- f- My computer was running slow, but I found out how to speed it up and make it safer.
- g- So there I was trying to find a job through the many online job posting sites but every time I tried to email my resume to a potential employer my computer would freeze. Or when I was searching for jobs this slow computer would make a simple search take an eternity.

→ EL PRESENTE PERFECTO

Uso: Usamos el Presente Perfecto (Present Perfect) para referirnos a sucesos relacionados con el pasado, pero cuando no sabemos o no estamos interesados en exactamente cuándo sucedió. También para mostrar una relación directa con el presente.

Estructura: **HAVE/HAS + PASADO PARTICIPIO**

e.g.: In the third millennium knowledge **has become** the most valuable asset.
Crime **has increased** as a consequence of poverty and marginalization.

PRÁCTICA

Lea el siguiente Abstract e identifique las estructuras verbales en Presente Perfecto. Traduzca al Español las correspondientes oraciones.

Ahora responda, ¿Cuál es el objetivo y contenido del trabajo de investigación indicado en este resumen?

Abstract

Mobile computers, such as cell phones and personal digital assistants (PDAs), have dramatically increased in sophistication. At the same time, the desire of consumers for portability limits battery size. As a result, many researchers have targeted hardware and software energy optimization. However, most of these techniques focus on compute-intensive applications rather than interactive applications, which are dominant in mobile computers. These systems frequently use graphical user interfaces (GUIs) to handle human-computer interaction. This paper is the first to explore how GUIs can be designed to improve system energy efficiency. We investigate how GUI design approaches should be changed to improve system energy efficiency and provide specific suggestions to mobile computer designers to enable them to develop more energy-efficient systems. We demonstrate that energy-efficient GUI (E²GUI) design techniques can improve the average system energy of three benchmarks (text-viewer, personnel viewer, and calculator) by 26.9, 45.2, and 16.4 percent, respectively. Average performance is simultaneously improved by 23.7, 34.6, and 19.3 percent, respectively.

→ EL PASADO PERFECTO

Uso: Usamos el Pasado Perfecto para referirnos a eventos ocurridos en un período de tiempo pasado anterior al correspondiente al del pasado simple.

Estructura: **HAD + PASADO PARTICIPIO**

e.g.: I passed my Maths exam last week, but before that I **had passed** my Chemistry exam.

→ EL PRESENTE/PASADO PERFECTO CONTINUO

Estructura: **HAVE/HAS/HAD + BEEN + -ING**

e.g.: You **have been studying** Information Systems Engineering for two or three years now.

She **had been doing** a different course of study before this one.

→ EL FUTURO

- BE GOING TO

Uso: Usamos "going to" para hablar sobre planes futuros o expresar intenciones.

e.g.: I'm **going to buy** a new computer next week.

They **are going to invite** two famous technologists for the conference.

Estructura: **AM/IS/ARE + GOING TO + FORMA BASE DEL VERBO PRINCIPAL**

- WILL

Uso: Usamos "will"

a) Cuando, en el momento en que estamos hablando, decidimos hacer algo en el futuro.

e.g.: I'll answer the question.

b) Para hacer especulaciones sobre cosas que no podemos controlar en el futuro.

e.g.: There will be great changes in our country over the next few years.

c) Para hablar sobre planes definidos en situaciones formales.

e.g.: Microsoft will launch new software very soon.

d) Cuando prometemos u ofrecemos hacer algo.

e.g.: I'll phone as soon as I get back.

e) Para pedir algo o pedir a alguien que haga algo.

e.g.: Will you help me, please?

f) Para dar órdenes.

e.g.: You will come back home before 12 pm!

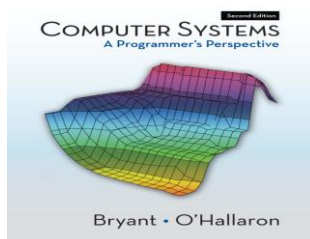
Estructura: **WILL + FORMA BASE DEL VERBO PRINCIPAL**

🔑 PRACTICA

I. Traducir al Español las siguientes oraciones:

- 1- In the future most libraries will be digital and users will have access to information by means of computers.
- 2- Experts say that the cost of maintaining a digital library is going to be lower than that of a traditional library. In fact, it is already lower.
- 3- Users will not have to leave their homes to get information from a library.
- 4- Will people abandon the use of traditional libraries or will they still prefer to keep going to the traditional building?
- 5- The university library is going to be reorganized to improve user's accessibility to latest information.

II. El siguiente texto corresponde a la introducción del Capítulo 1 del libro



Computer Systems: A Programmer's Perspective, 2/E (CS:APP2e) by Randal E. Bryant and David R. O'Hallaron, Carnegie Mellon University

<http://csapp.cs.cmu.edu/public/ch1-preview.pdf>

a- Lea el texto y enumere los objetivos planteados en el capítulo.

b- Luego ingrese a la dirección arriba señalada y realice una lectura de TODO el capítulo. Seleccione la sección (1 – 10) que más interesante le resulte y realice un resumen indicando el contenido de la misma.

Chapter 1

A Tour of Computer Systems

A *computer system* consists of hardware and systems software that work together to run application programs. Specific implementations of systems change over time, but the underlying concepts do not. All computer systems have similar hardware and software components that perform similar functions. This book is written for programmers who want to get better at their craft by understanding how these components work and how they affect the correctness and performance of their programs.

You are poised for an exciting journey. If you dedicate yourself to learning the concepts in this book, then you will be on your way to becoming a rare “power programmer,” enlightened by an understanding of the underlying computer system and its impact on your application programs.

You are going to learn practical skills such as how to avoid strange numerical errors caused by the way that computers represent numbers. You will learn how to optimize your C code by using clever tricks that exploit the designs of modern processors and memory systems. You will learn how the compiler implements procedure calls and how to use this knowledge to avoid the security holes from buffer overflow vulnerabilities that plague network and Internet software. You will learn how to recognize and avoid the nasty errors during linking that confound the average programmer. You will learn how to write your own Unix shell, your own dynamic storage allocation package, and even your own Web server. You will learn the promises and pitfalls of concurrency, a topic of increasing importance as multiple processor cores are integrated onto single chips.

In their classic text on the C programming language [58], Kernighan and Ritchie introduce readers to C using the `hello` program shown in Figure 1.1. Although `hello` is a very simple program, every major part of the system must work in concert in order for it to run to completion. In a sense, the goal of this book is to help you understand what happens and why, when you run `hello` on your system.

We begin our study of systems by tracing the lifetime of the `hello` program, from the time it is created by a programmer, until it runs on a system, prints its simple message, and terminates. As we follow the lifetime of the program, we will briefly introduce the key concepts, terminology, and components that come into play. Later chapters will expand on these ideas.

```
code/intro/hello.c
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("hello, world\n");
6 }
code/intro/hello.c
```

Figure 1.1: The `hello` program.

→ CONECTORES

Los conectores o conjunciones son palabras que usamos para unir dos o más frases o dos palabras dentro de la misma oración. Se los puede organizar en las siguientes categorías:

Copulativos: and, not only ... but also ..., not only ... but ... as well, both ... and ..., no sooner than

Disyuntivos: or, either ... or ..., neither ... nor ..., whether ... or ..., else, otherwise

Condicionales: if, whether, unless, provided, providing, as long as, in case, in case of

Concesivos: although, though, even though, even if, despite, in spite of, not even if, regardless of

Conclusivos: therefore, hence, thus, so, consequently

Continuativos: then, moreover, furthermore, besides, in addition to

Adversativos: but, however, nonetheless, yet, still, on the other hand, instead, instead of, on the contrary

Causales: because, for, because of, as a result of, due to, owing to, since, as

Comparativos: as, as ... as ..., not as ... as ..., not so ... as ..., just as ... so ..., as if, as though

Funcionales: so, so that, so as to, so as not to, in order to

e.g.:

- Technology contributes to society economic development. **Also**, it plays an important role in education.
- She passed the exam, **although** she had not studied./ **Although** she had not studied, she passed the exam.
- **In spite of /Despite** the few classes of English we had, we learnt a lot.
- We finished the class earlier **because** we were tired.
- We finished the class earlier **because of** our tiredness.
- The film was **so** good **that** I went to see it again.
- I am doing this course **to /in order to/ so as to** learn about educational technology.
- I gave you my e-mail **so that** you could send me your homework.

PRÁCTICA

El Capítulo 2: “Method of Separation of Variables” (p. 32 – 60)

(ftp://ftp.prenhall.com/pub/esm/sample_chapters/mathematics/haberman/pdf/ch02.pdf)

forma parte de la bibliografía correspondiente a la asignatura “Cálculo

Avanzado”, ¿A qué libro pertenece este capítulo? - Si no tiene la respuesta a esta pregunta, ingrese a la siguiente dirección y descúbralo:

http://books.google.com.ar/books?id=CTzvAAAAMAAJ&q=Method+of+Separation+of+Variables-Introduction-in+chapter+1+we+developed%22&dq=Method+of+Separation+of+Variables-Introduction-in+chapter+1+we+developed%22&hl=es&ei=CSEoTufzG8Kz0AGur9zBCg&sa=X&oi=book_result&ct=result&resnum=5&sqi=2&ved=0CEAQ6AEwBA

Ahora, identifique los conectores que aparecen en el texto y realice una breve traducción de las oraciones que contienen a los mismos.

Finalmente, responda las siguientes preguntas:

- 1- ¿Cuándo puede aplicarse directamente el método de separación de variables para resolver una ecuación diferencial parcial?
- 2- Defina operador lineal. Ejemplifique.
- 3- ¿cuándo una ecuación diferencial parcial lineal es homogénea? Ejemplifique.
- 4- Enuncie el Principio de Superposición.
- 5- Especifique cuáles son las ecuaciones diferenciales ordinarias que deben resolverse al aplicar el Método de Separación de Variables a la ecuación del calor

$$\frac{\partial u}{\partial t} = k \frac{\partial u}{\partial x^2}$$

$$\frac{\partial u}{\partial t} = k \frac{\partial^2 u}{\partial x^2}$$

- 6- ¿Cuáles son los valores propios (o autovalores) que producen soluciones triviales, es decir, funciones no nulas, para el problema a valores de borde (2.3.4)? (pág. 42) Indica las correspondientes auto-funciones.
- 7- ¿cuál es la expresión para la solución al problema de la ecuación del calor en una barra con temperatura cero en los bordes?
- 8- ¿Cuál es la expresión para la solución al problema de la ecuación del calor en una barra con los extremos aislados?